

5장_합성곱 신경망(CNN)

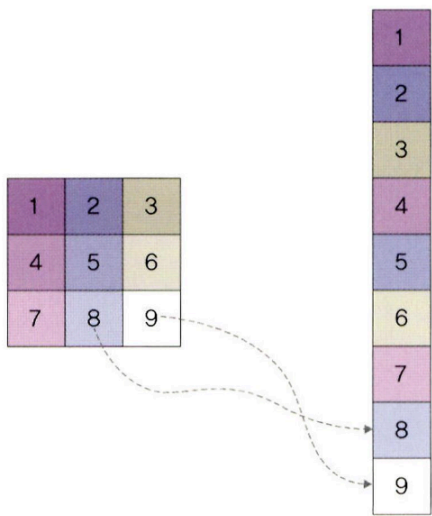
합성곱 신경망

- 합성곱 신경망은 이미지 전체를 한 번에 계산하는 것이 아닌 이미지의 국소적 부분을 계산함으로써 시간과 자원을 절약하여 이미지의 세밀한 부분까지 분석할 수 있는 신경망

합성곱층의 필요성

- 이전까지 익힌 DNN은 Fully Connected Layer로 구성 → Fully Connected Layer만으로 구성된 DNN의 입력 데이터는 1차원(배열)의 형태로 한정
 - 이미지는 1차원 형태의 데이터로 만들면, 공간적/지역적 정보가 손실됨
 - 별도의 추상화 과정 없이, 바로 raw data로 연산 과정에 놓이므로, 학습 효율성이 저하됨

▼ 그림 5-1 합성곱층 원리

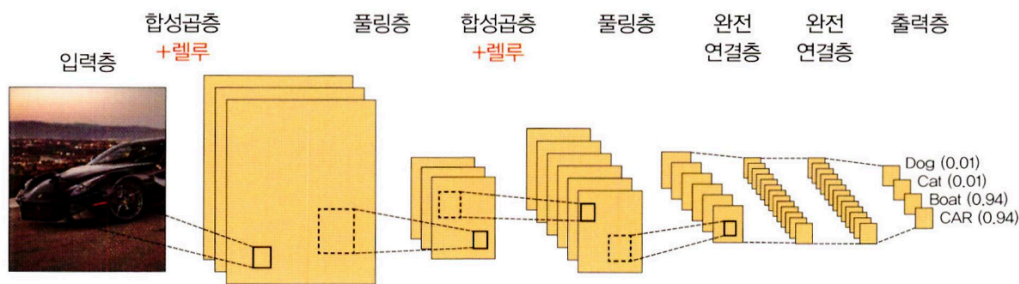


- CNN(convolutional Neural Network)는 공간적/지역적 정보를 유지한 상태로 학습 가능하도록 만든 모델**
- 이미지의 특정 부분은 전체에서 차지하는 비율은 작아도, 해당 이미지의 특징을 판단하기 위해, 중요한 특징일 수 있음
- 이를 확인하기 위해 **전체를 일정 비율로 잘라서 특정 부분을 확인하는 것이** 해당 특징을 확인하는데 효과적일 수 있음

합성곱 신경망 구조

- 음성 인식, 이미지/영상 인식에서 주로 사용되며, **다차원 배열 데이터**를 처리하도록 구성되어 컬러 이미지같은 다차원 배열 처리에 특화
- CNN은 **(1)이미지 특징을 추출하는 부분**과 **(2)클래스를 분류하는 부분**으로 구성됨

▼ 그림 5-2 합성곱 신경망 구조

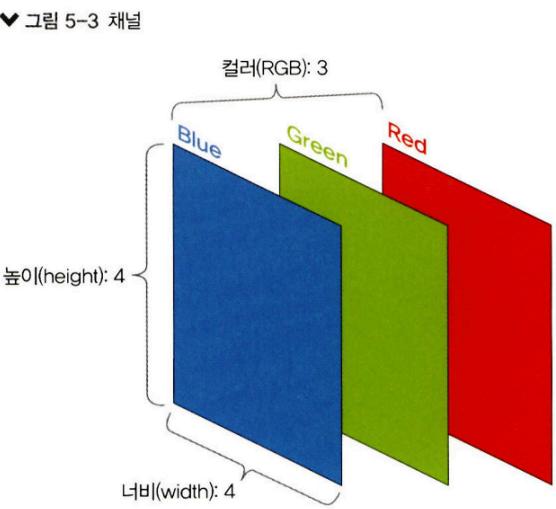


- (1) 이미지 특징을 추출하는 부분**
 - 합성곱 신경망은 **Convolution Layer와 Pooling layer를 여러 겹으로 쌓는 형태로 주요 특징 추출**(특성 벡터 = feature vector)
- (2) 클래스를 분류하는 부분**
 - 이전에 배운 **Fully Connected Layer**라고 생각하면됨 차원을 줄여서 분류를 만드는 것

- 추출된 주요 특성 벡터들은 완전 연결층을 거치면서 1차원 벡터로 변환 → 출력층에서 softmax 활성화 함수를 사용하여 최종 결과가 출력됨

1. 입력층

- 입력 이미지 데이터가 최초로 거치게 되는 계층 → 이미지는 Height, Width, Channel의 값을 갖는 3차원 데이터
- Channel**: 이미지 픽셀 값은 실수
 - 흑백 사진 : 각 픽셀 표현 위해 하나의 실수 필요 → 1개의 채널로 구성
 - 컬러 사진 : 각 픽셀에 RGB 3개의 실수 필요 → 3개의 채널로 구성 (ex) shape (4, 4, 3)



2. 합성곱층(convolutional layer)

- 입력 데이터에서 특성을 추출하는 역할 수행
- 특성 추출 : 입력 데이터에 **필터(filter) 또는 커널(kernel) “특징을 찾아내기 위한 공용 파라미터”**을 적용
- feature map(특성 맵)** : 필터 또는 커널은 이미지의 모든 영역을 훑으면서 특성을 추출하게 되는데 이런 추출된 결과물을 feature map 이라고 함

구분	정의	예시
채널	입력 이미지나 결과 이미지가 가진 층 (색상 정보 등)	RGB = 3채널
필터	합성곱에 사용되는 도구. 이미지에서 특징을 뽑음	3×3×3 크기의 필터
특성 맵	필터로부터 추출된 결과. 여러 개일 수 있음	2×2짜리 특성맵 여러 개

- stride**: 필터를 얼마만큼 움직여 주는가 → 기본적으로 1이지만, 테스트에 따라서 1 이상의 값으로 설정할 수도 있음

Stride와 Padding을 적용했을 때의 최종

- 입력 데이터 : W_1 (너비) $\times$ H_1 (높이) $\times$ D_1 (채널 또는 깊이)
- 하이퍼 파라미터
 - 필터 : FH (필터 높이), FW (필터 너비), K (필터 개수)
 - Stride 크기 : S
 - Padding 사이즈 : P
- 출력 데이터
 - 출력 너비(W_2) : $\frac{W+2P-FW}{S} + 1$
 - 출력 높이(H_2) : $\frac{H+2P-FH}{S} + 1$
 - 출력 깊이(D_2) : K

그레이스케일 이미지 합성곱 연산

▼ 그림 5-4 입력 이미지에 3×3 필터 적용



▼ 그림 5-5 입력 이미지에 필터가 1만큼 이동



▼ 그림 5-6 입력 이미지에 필터가 1만큼 두 번째 이동



▼ 그림 5-7 입력 이미지에 필터가 1만큼 세 번째 이동



▼ 그림 5-8 입력 이미지에 필터가 1만큼 네 번째 이동



▼ 그림 5-9 입력 이미지에 필터가 1만큼 마지막으로 이동

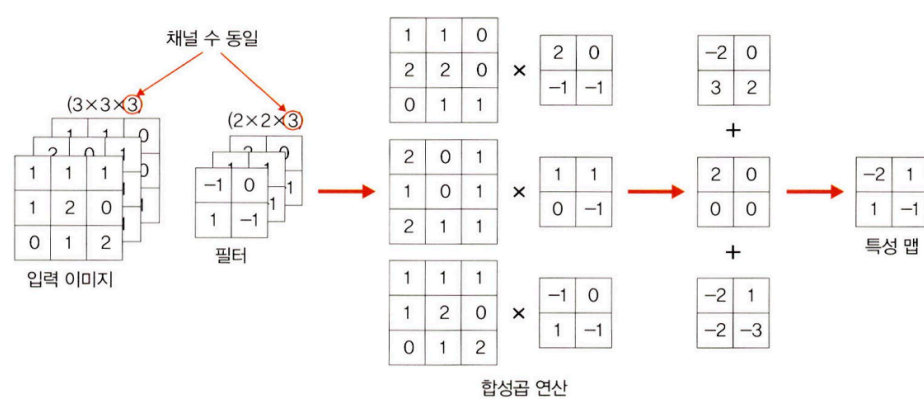


- 입력 이미지와 필터를 포개 놓고 대응되는 숫자끼리 곱한 후 모두 더함
- kernel은 stride 간격만큼 순회하면서 모든 입력 값과의 합성곱 연산으로 새로운 특성 맵을 만들
- kernel과 stride의 상호 작용으로 원본 (6, 6, 1) 크기가 (4, 4, 1) 크기의 특성 맵으로 줄어들

컬러 이미지 합성곱 연산

- 필터 채널이 3개이며, RGB 각각에 서로 다른 가중치로 합성곱을 적용한 후 결과를 더해줌
- ★ 필터 채널이 3이라고 해서 필터 개수도 3개라고 오해하기 쉬운데, 필터 개수는 한 개

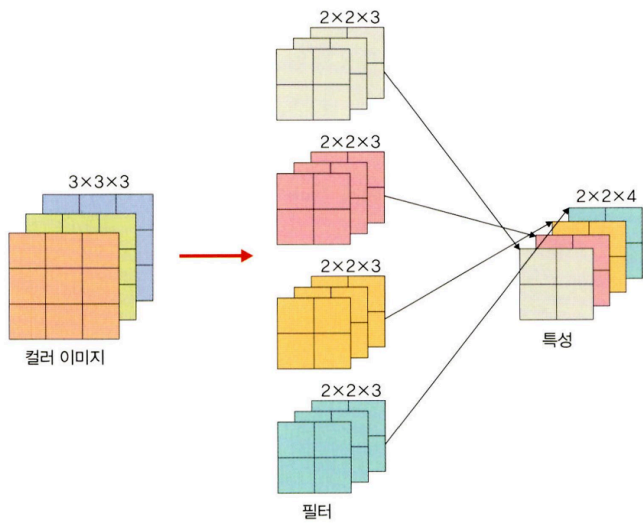
▼ 그림 5-10 컬러 이미지 합성곱



필터가 2개 이상인 합성곱의 형태

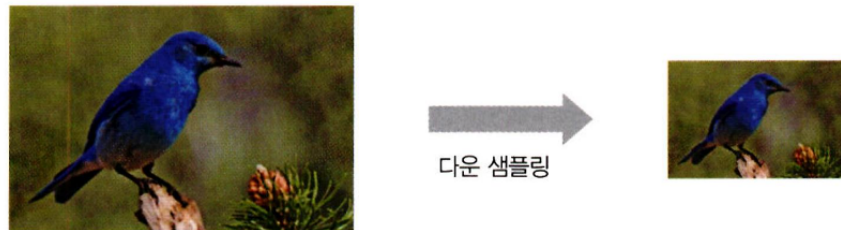
- **그니까 3채널에서 4채널로 늘어난 거지?"**
 - 처음 입력 이미지의 채널은 보통 RGB니까 3채널 → 하지만 합성곱(convolution) 후에는 필터 개수만큼 채널 수가 바뀜 (ex) 필터를 4개 쓰면 출력은 4채널
- **그리고 그 채널의 원래 의미는 R, G, B를 의미하는 거였다면, 필터를 거치면서 다른 것을 의미하게 된 거고?"**
 - 입력 이미지의 채널(R, G, B)은 색상 정보였지만 → 합성곱 후의 채널들은 필터가 추출한 '특징 정보'(ex : 엣지, 곡선, 텍스처, 밝기 변화, 선명도 등) 으로 채널이 의미하는 게 바뀜

▼ 그림 5-11 필터가 2 이상인 합성곱



3. 풀링층

- Convolution Layer에서 얻어진 출력값에서 특징을 뽑아내는 과정 → 일반적으로 데이터 차원 수를 줄이는 등의 작업은 데이터의 특징을 추출하는 과정으로 볼 수 있음
- Pooling은 각 Feature Map의 사이즈를 줄이며, 특징을 뽑아냄**
 - 특성 맵의 차원을 다운 샘플링하여 연산량을 감소시키고, 주요한 특성 벡터를 추출하여 학습을 효과적으로 할 수 있게 함



1. **max pooling** : 대상 영역에서 최댓값을 추출

2. **average pooling** : 대상 영역에서 평균을 반환

- 대부분의 CNN에서는 max pooling이 사용됨 → average pooling은 각 커널 값을 평균화 시켜 중요한 가중치를 갖는 값의 특성이 희미해질 수 있기 때문

• 입력 데이터 : $W_1 \times H_1 \times D_1$

• 하이퍼파라미터

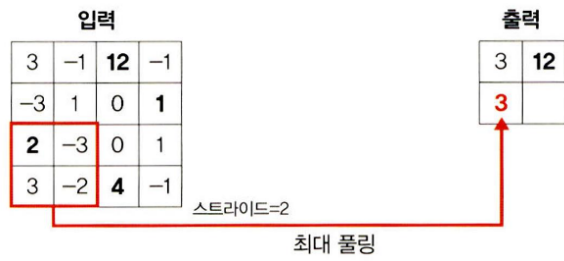
- 필터 : FH, FW
- 스트라이드 : X

• 출력 데이터

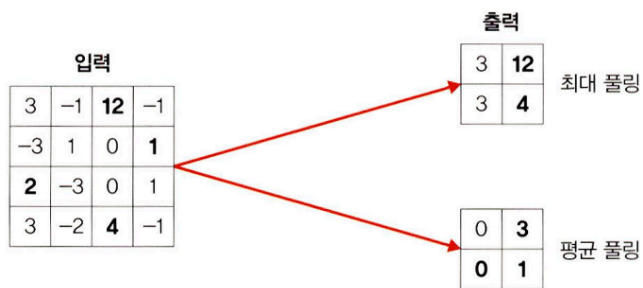
- $W_2 = \frac{W_1 - FW}{S} + 1$
- $H_2 = \frac{H_1 - FH}{S} + 1$
- $D_2 = D_2$

최대 풀링의 연산 과정

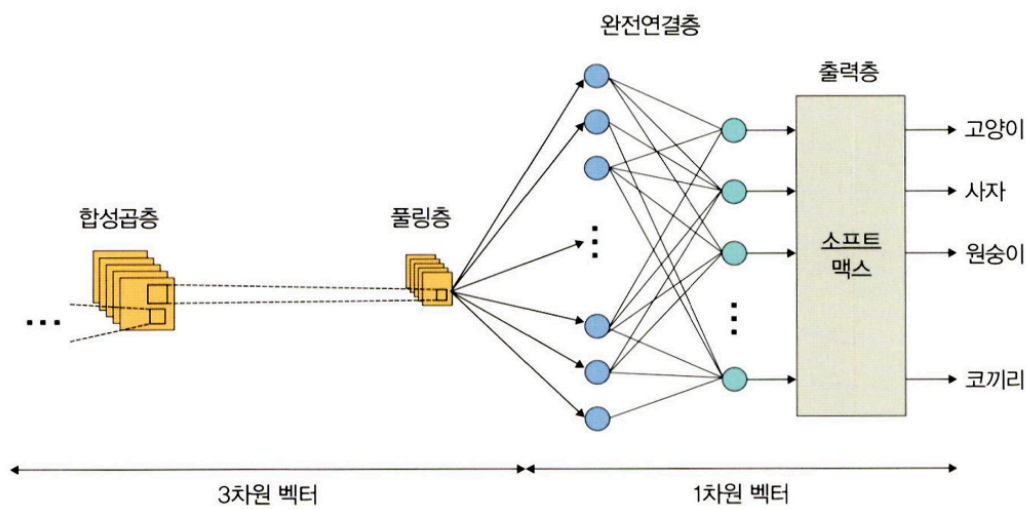




최대 풀링과 평균 풀링 비교



4. 완전연결층(fully connected layer)



- 클래스를 분류하는 부분
- convolution layer와 pooling layer를 거치면서 차원이 축소된 특성 맵은 최종적으로 fully connected layer로 전달됨
- 이 과정에서 이미지는 3차원 벡터에서 1차원 벡터로 만들어 줘야함 = **Flatten(Vectorization)**

5. 출력층(output layer)

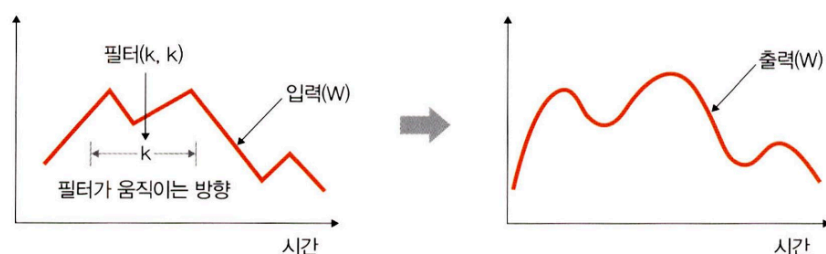
- softmax activation function이 사용되는데, 입력받은 값을 0~1 사이의 값으로 출력함
- 따라서 마지막 출력층의 softmax 함수를 사용하여 이미지가 각 label에 속할 확률 값이 출력되며, 이때 가장 높은 확률 값을 갖는 label이 최종 값으로 선정됨

1D, 2D, 3D 합성곱

- 합성곱은 이동하는 방향의 수와 출력 형태에 따라 1D, 2D, 3D 로 분류할 수 있음

1D 합성곱

▼ 그림 5-19 1D 합성곱

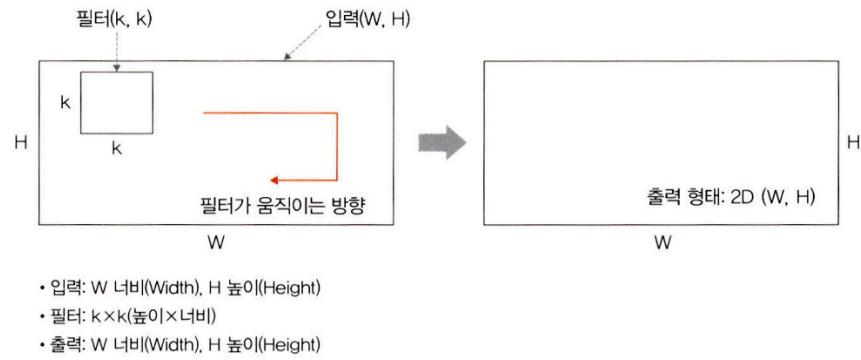


- 입력: W 너비(Width)
- 필터: k×k(높이)×너비
- 출력: W 너비(Width)

- 필터가 시간을 축으로 좌우로만 이동할 수 있는 합성곱 → 입력(W)과 필터(k)에 대한 출력은 W
 - (ex) 입력 $[1, 1, 1, 1]$ 필터 $[0.25, 0.5, 0.25]$ → 출력 $[1, 1, 1]$
 - 그래프 곡선을 완화할 때 많이 사용됨

2D 합성곱

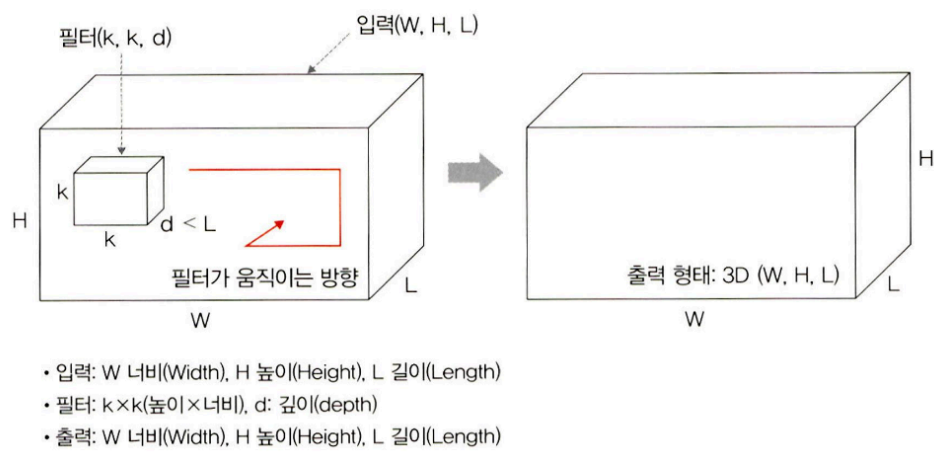
▼ 그림 5-20 2D 합성곱



- 필터가 다음 그림과 같이 방향 두 개로 움직이는 형태
 - 입력(W, H)과 필터(k, k)에 대한 출력은 (W, H) → 출력 형태는 2D 행렬

3D 합성곱

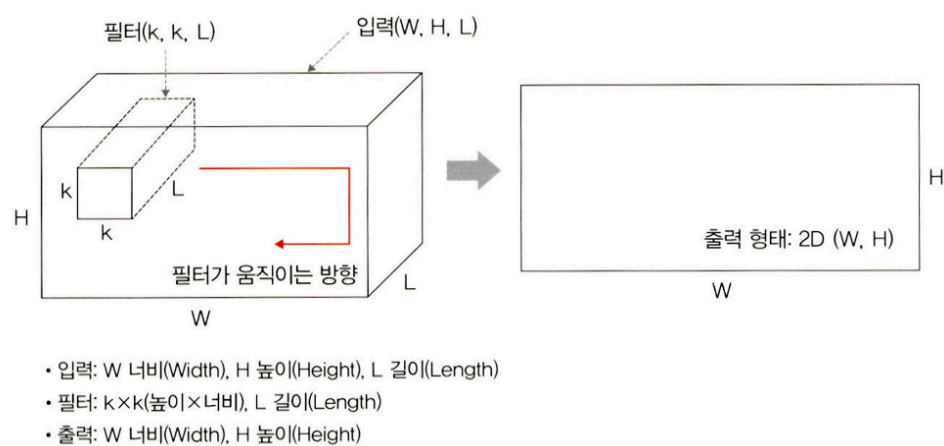
▼ 그림 5-21 3D 합성곱



- 3D 합성곱은 필터가 움직이는 방향이 세 개
 - 입력(W, H, L)에 대해 필터(k, k, d)를 적용하면 출력으로 (W, H, L)을 갖는 형태
 - 출력은 3D 형태 → 이때 $d < L$ 을 유지하는 것이 중요

3D 입력을 갖는 2D 합성곱

▼ 그림 5-22 3D 입력을 갖는 2D 합성곱



- 입력이 $(224 \times 224 \times 3, 112 \times 112 \times 32)$ 와 같은 3D 형태임에도 출력 형태가 3D가 아닌 2D 행렬을 취하는 것
- 필터에 대한 길이(L)가 입력 채널의 길이(L)와 같아야 하기 때문에 이와 같은 합성곱 형태가 만들어짐
 - 즉, 입력(W, H, L)에 필터(k, k, L)를 적용하면 출력은 (W, H)가 됨 → 출력 형태는 2D 행렬

- 대표적 사례 : LeNet-5와 VGG
-

1X1 합성곱

- 1X1 합성곱은 3D 형태로 입력 → 입력(W, H, L)에 필터(1, 1, L)를 적용하면 출력은 (W, H)
 - 1X1 합성곱에서 채널 수를 조정해서 연산량이 감소되는 효과
 - 대표적 사례 : GoogleNet